Who Can See the AI Code? Detection Blind Spots and the Limits of Prevalence Estimates on GitHub

Maulik Kaushik
Independent Research · github.com/MUCKSUCK-Oss/ai-code-scanner · August 2026

ABSTRACT

Large-scale studies of AI-generated code on GitHub identify it through Git metadata: `Co-authored-by` trailers, agent bot accounts, and tool-specific author emails. That approach finds only the code that announced itself. We apply metadata detection and an independent content-based classifier to the same 9,982 repositories, spanning popular projects (≥1000 stars), new low-visibility projects (<10 stars, created after 2023), and a control cohort of 3,333 repositories with no push since 2021. The metadata method yields **zero false positives** across the control; the content classifier yields 0.99%. Critically, the two methods agree on **10 repositories out of 9,982**, and content detection identifies 106 repositories containing AI-written code with no Git marker of any kind — a 13% addition to what metadata finds, and a population absent from every published prevalence estimate. Testing whether AI use predicts adoption, we find a statistically significant but negligible association in established projects ($r = +0.131$) and none in new ones ($r = +0.016$, $n = 3{,}323$). We further document three measurement artifacts that produced confident but incorrect results before detection, one of which is embedded in the standard method: binary AI-presence flags measure repository activity as much as AI adoption, and reverse the apparent cohort difference when corrected.

1 Introduction

AI coding assistants have moved from novelty to infrastructure. GitHub's Octoverse 2025 reports 230 new repositories created every minute. Maintainers report the consequences directly: in January 2026 the curl project terminated its bug bounty programme after reporting that roughly 20% of submissions were AI-generated noise against approximately 5% genuine, describing the load as a denial-of-service on reviewer capacity [4].

A body of empirical work has grown up to measure it. Liu et al. [1] analysed 304,362 AI-authored commits across 6,275 repositories; GitClear [2] examined 211 million lines of changed code; a Carnegie Mellon study [3] tracked 807 repositories adopting Cursor. These studies converge on a finding: AI-generated code accumulates technical debt that persists rather than being refactored away.

They also share a methodological premise that has not been examined. Each identifies AI code by locating code that *labelled itself* — through commit trailers, bot accounts, or tool-specific metadata. A developer who pastes output from a chat interface into an editor leaves no such label. Whatever that population contains, it has never been counted, and it is precisely the population most often invoked in arguments about low-quality AI output.

1.1 Contributions

This paper makes three contributions. First, we quantify the gap between metadata-based and content-based detection on a common sample, establishing that the two methods are near-disjoint rather than overlapping. Second, we test whether the quantity of AI use in a repository predicts its adoption, controlling for repository age and cohort. Third, we document three measurement artifacts encountered during the study, including one present in the standard detection criterion, each of which produced a confident and incorrect result before being identified.

2 Related Work

Quality and technical debt. Liu et al. [1] performed commit-level differential static analysis on verified AI-authored commits, identifying 484,606 distinct issues of which 24.2% survived to repository HEAD, with security issues persisting at 41.1%. GitClear [2] reported duplicated code blocks rising sharply and code churn approximately doubling relative to a pre-AI baseline. Yang et al. [3] found static analysis warnings increasing roughly 30% and complexity roughly 41% following Cursor adoption.

Detection methodology. Liu et al. [1] identify AI commits through actor logins, author emails, author names, and co-authorship trailers, covering 29 tools that leave identifiable traces. A multi-method census

of 180 million repositories [5] concludes that no single detection approach captures all AI-generated code, and names improved content-based heuristics as future work.

Population coverage. Existing studies apply inclusion thresholds — Liu et al. [1] require ≥100 stars — that exclude the low-visibility population by construction. Our new-repository cohort is drawn specifically from below that threshold.

3 Method

3.1 Sampling

We drew 9,999 repositories in three cohorts of 3,333. **Popular**: ≥1000 stars, pushed since 2024. **New**: <10 stars, created after 2023. **Control**: ≥50 stars, created 2012–2020, *with no push since January 2021*.

The push constraint on the control is essential and was added after an initial run. Selecting on creation date alone returns repositories that are old but still actively developed; because we scan the most recent 250 commits, those commits date from 2024–2026 and contain genuine AI contributions. In that flawed configuration, 20.1% of the "pre-LLM" control was flagged. With the constraint applied, the control has a median dormancy of 5.6 years and every line in it necessarily predates LLM assistance.

Sampling draws an even quota from every (language, year) cell across Python, JavaScript, TypeScript, Java, and Go. An earlier greedy implementation returned a cohort that was entirely Python and entirely 2009–2012, which would have confounded the comparison, since older repositories are both more popular and definitionally more human-written.

3.2 Metadata detection

For each repository we scan the most recent 250 commits for markers from six assistants (Claude, GitHub Copilot, Cursor, Devin, Gemini, Codex), matching against co-authorship trailers, agent author emails, and bot account names. This mirrors prior work, making our metadata arm directly comparable.

We record two distinct quantities: whether *any* marker is present, and the *share* of commits carrying one. Section 5.1 shows these disagree about which cohort uses more AI. We additionally parse each AI commit's diff statistics, counting a commit only when it modified source files; documentation-only commits are recorded separately.

3.3 Content detection

Files are scored 0–100 on structural fingerprints, partitioned by specificity. *Weak* signals — comment-to-code ratio, docstring coverage, type-hint saturation, generic identifiers — are equally consistent with disciplined engineering. *Strong* signals are structural artifacts of generation: conversational boilerplate, uniformly padded box-drawing section banners, sequentially numbered step comments, self-narrating commentary about the editing process, filename-description scaffolding headers, and rigid documentation templates repeated across every function.

When no strong signal fires, the score is capped at 40. This rule was introduced after `psf/requests`, released years before LLMs existed, produced a maximum-score false positive on docstring density and comment ratio alone. Repository scores are the LOC-weighted mean of file scores.

3.4 Adoption

We measure adoption as stars and forks per month since creation, log-scaled to prevent a small number of very popular repositories from dominating, and compared strictly within age bands. Age is the primary confound: older repositories have had longer to accumulate stars and are more likely to predate AI tooling.

4 Validation

Before any comparative result can be interpreted, the instrument must be shown not to fire on human-written code. Figure 1 reports AI-authored commit share by cohort. The control registers exactly zero.

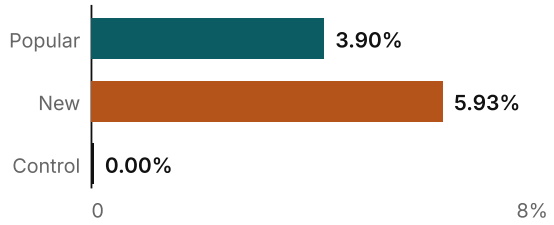


Figure 1. Share of the most recent 250 commits that are AI-authored and modified source code. The pre-LLM control (n = 3,333, median 5.6 years dormant) registers zero, as it must.

Table 1. False-positive rates on the pre-LLM control cohort (n = 3,333).

Method	Flagged	Rate
Git metadata	0	0.00%
Content classifier	33	0.99%

5 Results

5.1 Presence is not prevalence

Applying the criterion used in prior work — a repository counts as AI-involved if any marker appears — yields 55.6% for popular repositories against 26.2% for new ones, implying professional projects use substantially more AI. Measuring the share of commits that are AI-authored reverses the ordering: 3.90% against 5.93%.

Nothing about developer behaviour explains this. A repository with 250 commits and forty contributors picks up an AI commit the moment any one of those contributors uses a tool once. In an earlier sample, three repositories were flagged on a single commit out of 250, including a large professional IDE at 0.4%. So the criterion tracks contributor count and commit volume, and both of those rise with popularity by construction.

> **RECOMMENDATION**
> Prevalence work should report the share of commits that are AI-authored, not the proportion of repositories containing any AI commit. On identical data the two measures point in opposite directions.

5.2 The two methods are near-disjoint

Table 2 reports the joint distribution. Of 908 repositories flagged by either method, the two agreed on ten.

Table 2. Joint detection outcomes across 9,982 repositories.

Outcome	Repos	Share
Metadata only	792	7.9%
Content only	106	1.1%
Both methods	10	0.1%
Neither	9,074	90.9%

Those 106 repositories carry generation fingerprints in the code and no Git marker at all. Every metadata-based estimate in the literature misses them, and they add 13% to the metadata count. Conversely, 792 repositories carry AI markers while their code reads as human-written — consistent with AI output that was reviewed and edited before merging.

5.3 AI use and adoption

Table 3 reports the correlation between AI score and log-scaled adoption within age bands. Table 4 repeats the test within cohorts, which is the stricter control.

Table 3. AI score against adoption, within age bands (popular and new cohorts, n = 6,649).

Age band	n	r	crit.
0–6 months	909	−0.053	0.066
6–12 months	600	+0.026	0.082
1–2 years	880	+0.105	0.067
2–5 years	1,836	+0.116	0.047
5+ years	2,424	+0.042	0.041

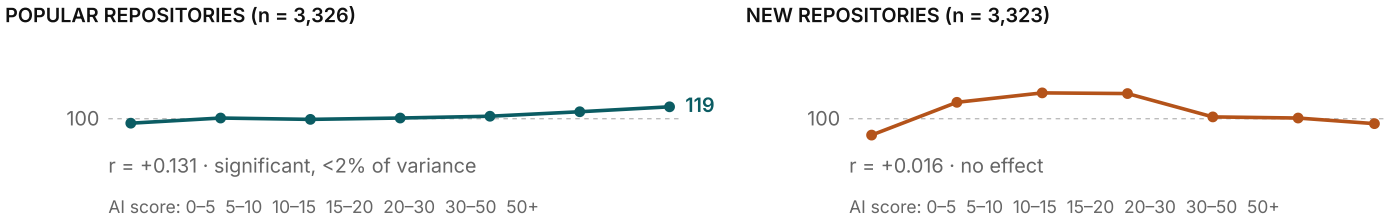


Figure 2. Adoption plotted against AI score within each cohort, indexed so that 100 equals that cohort's own mean. Indexing allows two groups with very different absolute adoption (4.41 against 0.12) to be compared on one axis. In popular repositories the line rises gently; in new repositories it is flat. Pooling the two cohorts instead produces an apparent inverted-U, which is an artifact of composition rather than a real relationship (Section 6.1).

Table 4. The same test within cohorts, removing cohort membership as an explanation.

Cohort	n	r	Verdict
Popular	3,326	+0.131	significant
New	3,323	+0.016	no effect

In established projects the association is statistically significant and substantively negligible: r = +0.131 accounts for under 2% of variance in adoption. In new projects — where a project built primarily with AI would be found — there is no association at all.

The more plausible causal direction runs opposite to the one usually asserted. Active, well-staffed projects adopt tooling of every kind, AI included. Nothing in these data supports the claim that AI use causes a project to succeed, and nothing supports the claim that it causes a project to fail.

WHAT THIS EXCLUDES

At n = 3,323 the significance threshold is r = 0.035. A substantial effect in either direction would have surfaced. The proposition that AI-generated code measurably harms a project's prospects — and its converse — is unsupported at this sample size.

5.4 Secondary results

Table 5. Additional measurements.

Measure	Result
Duplicate file groups	1,606
AI score vs. defect density (n = 2,760)	r = −0.125
Abandonment rate (no push in 180 days)	3%
Repositories with any Claude marker	2,168
Repositories with any Copilot marker	932
Repositories with any Cursor marker	552

Identical normalised source files were found across 1,606 distinct groups of repositories, indicating that literal code duplication between projects is common at this scale. The negative correlation between AI score and defect density is significant but small, and is likely confounded by project maturity in the same manner as Section 6.3.

6 Measurement Artifacts

Three of our measures produced confident, publishable-looking results that were wrong. We report them for two reasons: the first is built into the field's standard criterion, and each surfaced only through a robustness check the initial analysis had not included.

6.1 Cohort mixing

A pooled analysis of adoption against AI score initially produced r = +0.271 in the 2–5 year band, apparently indicating that AI-heavy projects attract more users. In a pilot the band comprised 13 popular and 61 new repositories. Because popular repositories score higher on both variables, mixing them manufactures a correlation. Within each cohort separately the effect vanished (r = +0.246 and +0.084, neither significant). This pattern recurred four times during the study.

6.2 An AI commit is not AI code

A commit carrying a co-authorship trailer may have modified only a README. Parsing diff statistics removed 15% of flagged commits as documentation-only and reduced one repository from apparent AI authorship to zero.

6.3 Engineering practice is not value

An initial measure of "slop" scored repositories on missing tests, CI configuration, and licences. Applied to the control cohort it returned 24.9 — higher than the new AI-heavy cohort at 23.8. Human code written in 2015 and subsequently abandoned scored as more slop-like than anything AI produced. Decomposition showed 81% of the score derived from professional-practice signals rather than substance. The measure was replaced by adoption.

7 Limitations

1. **Observational.** No causal claim is made or supported.
2. **The content classifier is weak.** Evaluated against metadata-labelled ground truth it achieved AUC = 0.604, marginally above chance. It detects raw generated output, not AI code that has been reviewed and edited. Its 99 unique identifications are genuine, but it is not a general-purpose detector.
3. **Quality analysis is Python-only.** Pylint, Bandit and radon do not cover JavaScript, restricting that arm to 2,760 repositories.
4. **History depth is capped at 250 commits**, rendering earlier AI use in long-lived repositories invisible.
5. **Adoption imperfectly proxies value.** Stars reward visibility and timing as well as utility.

8 Conclusion

Across 9,982 repositories, the two available methods for detecting AI-generated code agreed on ten of them. The metadata method — the foundation of every published prevalence estimate — correctly registered nothing across 3,333 repositories written before LLMs existed, and missed 106 repositories that demonstrably contain generated code today.

On the question driving the wider argument, the answer is dull. AI use does not predict whether a new project finds users. In established projects the association is positive, significant, and small enough to be uninteresting. At this sample size a real effect in either direction would have surfaced.

Neither side of the argument survives contact with the data. A narrower claim does: prevalence estimates built on self-declared AI code measure which tools developers picked as much as how much they used them, and the repositories most often accused of pro-

ducing worthless output are the ones these methods can least see.

References

1. Liu, Y., Widyasari, R., Zhao, Y., Irsan, I. C., and Lo, D. Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild. arXiv:2603.28592, 2026.
2. GitClear. AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones. Technical report, 2025.
3. Empirical study of Cursor adoption across 807 open-source repositories. Carnegie Mellon University, 2025.
4. curl project. Ending the bug bounty programme. January 2026.
5. Detecting AI Coding Agents in Open Source: A Validated Multi-Method Census of 180 Million Repositories. arXiv:2606.24429, 2026.
6. A Large-Scale Empirical Study of AI-Generated Code in Real-World Repositories. arXiv:2603.27130, 2026.

Data and analysis code: github.com/MUCKSUCK-Oss/ai-code-scanner · Raw records: `paper/data/final_n10000.json` (9,982 repositories) · All figures reproducible via `dual_detect.py`, `correlate.py`, `value_audit.py`.